

IST 602: Web Technologies and Standards - Detailed Exam Solutions

Course Code: IST602

Institution: University of Buea, Faculty of Science

Semester: Second Semester 2024/2025

Instructor: Denis Nkwuemeyin

Level: Master's Program

Time Allowed: 3 Hours

QUESTION 1: XML Documents and Schemas (25 Marks)

Part a) XML Syntax Rules and Document Validation (4 marks)

i) Three Syntax Rules for XML Conformity

1. **Element Structure Rule:** All XML elements must be properly nested. An element opened inside another must be closed before the parent element closes. This ensures hierarchical consistency and prevents overlapping tags that would create ambiguous document structures.
2. **Well-Formedness Rule:** XML documents must contain exactly one root element that encompasses all other elements. Additionally, all tags must be properly paired (opening and closing), and attribute values must be enclosed in either single or double quotes.
3. **Character Encoding and Declaration Rule:** XML documents should include an XML declaration specifying the version (typically 1.0) and character encoding (UTF-8, UTF-16, etc.). This ensures proper parsing across different systems and prevents character interpretation errors.

ii) Definition of a Well-Formed XML Document

A **well-formed XML document** is a document that adheres to all XML syntax rules, regardless of its validity. Specifically:

- It contains exactly one root element
- All elements are properly nested with no overlapping tags
- All tags have both opening and closing forms (or are self-closing)
- All attribute values are properly quoted
- Special characters are properly escaped (using entity references like `<`, `&`, etc.)
- The document includes a proper XML declaration (optional but recommended)

Key Distinction: Well-formed means syntactically correct; it does NOT guarantee semantic correctness.

iii) Definition of a Valid XML Document

A **valid XML document** is a well-formed XML document that additionally conforms to rules defined in an external schema (DTD or XML Schema). Validation ensures:

- The document structure matches the defined schema
- Elements contain the correct data types
- Required elements are present
- Attribute constraints are satisfied
- Element ordering and occurrence rules are followed

Relationship: A valid document must be well-formed, but a well-formed document is not necessarily valid.

iv) Analysis of XML Documents in Figures 1 and 2

Figure 1 Analysis:

```
<note>
  <to>Tove</to>
  <from>Jane</from>
  <heading>Reminder</heading>
  <body>Don't forget me today!</body>
</note>
```

- **Status:** WELL-FORMED ✓
- **Reasoning:**
 - Has single root element `<note>`
 - All elements properly nested
 - All tags properly closed
 - Follows XML structure rules

Figure 2 Analysis:

```
<?xml version="1.0"?>
<note>
  <to>Tove</to>
  <heading>Reminder</heading>
  <body>Don't forget me today!</body>
</note>
```

- **Status:** WELL-FORMED ✓
- **Reasoning:**
 - Includes XML declaration
 - Single root element
 - Proper nesting and tag closure
 - Missing `<from>` element (but this doesn't affect well-formedness—only validity)

Conclusion: Both documents are well-formed. Neither would be valid unless they conform to a predefined schema requiring specific elements like `<from>`.

Part b) DTDs and XML Schemas (6 marks)

i) What are DTDs and Schemas Used For

DTDs (Document Type Definitions):

- Define the structure and grammar of XML documents
- Specify which elements are allowed and their relationships
- Define element attributes and their constraints
- Control element ordering and occurrence (cardinality)
- Validate XML documents against the defined rules
- Can be declared internally or externally

XML Schemas (W3C XML Schema):

- More powerful than DTDs
- Define data types for elements and attributes
- Support namespace management
- Enable complex type definitions and inheritance
- Provide superior validation capabilities
- Offer better reusability through modularization

Primary Purpose: Both enforce structural and semantic constraints to ensure XML documents meet specific business requirements.

ii) Three Problems with XML Schemas**1. Complexity and Steep Learning Curve:**

- XML Schema syntax is verbose and complex
- Requires deep understanding of type systems
- Large schema files become difficult to maintain
- Migration from DTD to Schema requires significant rework
- Development time increases substantially

2. Namespace Management Challenges:

- Namespace handling in XML Schema is intricate and error-prone
- Prefix declarations and scope management add complexity
- Namespace conflicts can cause validation failures
- Interoperability issues between different namespace implementations

3. Limited Expressiveness in Certain Domains:

- Cannot easily express certain real-world constraints (e.g., co-occurrence rules)
- Circular dependency modeling is problematic
- Cannot enforce business logic beyond structural validation
- Some constraints require post-schema-validation application logic

iii) Reusable XSD Rules Defined in DTD

Based on the DTD provided, reusable rules include:

```
<!-- Element Declaration Rules -->
<!ELEMENT bibliography [
    (book)+
]>
<!ELEMENT book [
    (author | (#PCDATA REQUIRED))+
]>

<!-- Attribute Declaration Rules -->
<!ATTLIST book
    ISBN CDATA #REQUIRED
    year CDATA #IMPLIED
    section* CDATA #FIXED
]>

<!-- Entity Rules -->
<!ENTITY copyright
```

```
"© 2024"
>
<!ENTITY footer
  SYSTEM "footer.xml"
>
```

Reusable Components:

- Content models for standardized structures
- Attribute patterns applicable across multiple elements
- Entity definitions for common repeated content
- Cardinality rules (*, +, ?, none) for occurrence constraints

Part c) XML Representation of Book Data (4 marks)

Original Table Data:

Book	Field	Attribute	Field	Attribute	Field	Attribute	Field	Attribute	Field	Attribute
	Title	Language	Author		Year		Price		Category	
1	Every day Italian	English	Giada De Laurentiis		2005		30.00		Cooking	
2	Harry Potter	English	J.K. Rowling, James McGovern		2005		29.99			
3	XQuery Kick Start	English	Per Bothner, Kurt Caple		2003		49.99		Web	

XML Representation:

```
<?xml version="1.0" encoding="UTF-8"?>
<bookstore>
  <book id="1">
    <title language="English">Every day Italian</title>
    <author>Giada De Laurentiis</author>
    <year>2005</year>
    <price>30.00</price>
    <category>Cooking</category>
  </book>

  <book id="2">
    <title language="English">Harry Potter</title>
    <author>J.K. Rowling</author>
    <author>James McGovern</author>
    <year>2005</year>
    <price>29.99</price>
  </book>

  <book id="3">
    <title language="English">XQuery Kick Start</title>
```

```
<author>Per Bothner</author>
<author>Kurt Caple</author>
<year>2003</year>
<price>49.99</price>
<category>Web</category>
</book>
</bookstore>
```

Design Rationale:

- Root element `<bookstore>` contains all books
- Each book has a unique `id` attribute
- Language is an attribute of title (metadata about the field)
- Multiple authors are represented as separate `<author>` elements
- Price stored as string (could be typed in Schema)
- Optional category for books without category information

QUESTION 2: TypeScript (25 Marks)

Part a) TypeScript Fundamentals (2+2 marks)

i) What is a Transpiler and What TypeScript Compiles To

A **transpiler (source-to-source compiler)** is a tool that translates code written in one programming language into equivalent code in another language at a similar abstraction level.

TypeScript Compilation Process:

- TypeScript is a strict superset of JavaScript that adds static type annotations
- The TypeScript compiler (`tsc`) reads `.ts` files and transpiles them into `.js` (JavaScript)
- During transpilation:
 - Type annotations are stripped (they serve only compile-time purposes)
 - Advanced ES2015+ syntax is converted to ES5/ES3 for browser compatibility (based on target)
 - Type checking occurs (errors prevent successful compilation)
 - Source maps are generated for debugging

Output Characteristics:

- Pure JavaScript with no type information
- Compatible with all JavaScript environments
- May include polyfills and runtime helpers
- Functionally equivalent to input TypeScript

ii) TypeScript to JavaScript Transformation Example

```
// TypeScript Input
function greet(name: string): void {
    console.log(`Hello, ${name}`);
}

const age: number = 25;
const isStudent: boolean = true;
```

Compiles to:

```
// JavaScript Output (ES5)
"use strict";
function greet(name) {
    console.log("Hello, " + name);
}
var age = 25;
var isStudent = true;
```

Changes Made:

- Type annotations removed (`: string, : number, : boolean`)
- Arrow functions converted to function expressions if targeting ES5
- `const` may convert to `var` depending on target
- Template literals converted to string concatenation
- Type information completely eliminated

Part b) Function Validity Assessment (2+4 marks)

i) Function Parameter and Type Analysis

```
(i) function myFunc(x:number, y:string): string { }
(ii) function myFunc(pt: { x: number; y: number }): { }
(iii) function myFunc(x:number, y:string) { }
(iv) function myFunc(x:number, y): number { }
```

Validity Assessment:

Function	Valid?	Parameters	Return Type	Analysis
(i)	YES ✓	x: number, y: string	string	Properly typed, clear contract
(ii)	YES ✓	pt: object	implicit any	Valid, but missing explicit return type
(iii)	YES ✓	x: number, y: string	implicit any	Valid; return type defaults to any
(iv)	PROBLEMATIC ⚠	x: number, y: any	number	y lacks type annotation; ambiguous

Function Signatures Clarified:

```
// (i) - Fully typed
function myFunc(x: number, y: string): string {
    return x + y; // Type error in actual implementation
}

// (ii) - Object parameter
function myFunc(pt: { x: number; y: number }): void {
    console.log(pt.x, pt.y);
}

// (iii) - Implicit return type
function myFunc(x: number, y: string): any {
    // Return type not specified, defaults to 'any'
}
```

```

}

// (iv) - Incomplete typing
function myFunc(x: number, y): number {
  // 'y' is implicitly 'any' - violates strict mode typing
  return x;
}

```

ii) Parameter and Return Type Details

Complete Specification for Each Function:

Function (i): `function myFunc(x: number, y: string): string`

- **Parameters:**
 - `x`: type `number` (required)
 - `y`: type `string` (required)
- **Return Type:** `string`
- **Function Type:** `(x: number, y: string) => string`

Function (ii): `function myFunc(pt: { x: number; y: number }): any`

- **Parameters:**
 - `pt`: object with properties `x: number, y: number`
- **Return Type:** `any` (implicit; should be specified)
- **Recommended:** `function myFunc(pt: { x: number; y: number }): void`

Function (iii): `function myFunc(x: number, y: string): any`

- **Parameters:**
 - `x`: type `number`
 - `y`: type `string`
- **Return Type:** `any`
- **Issue:** Missing explicit return type leads to type inference

Function (iv): `function myFunc(x: number, y: any): number`

- **Parameters:**
 - `x`: type `number`
 - `y`: type `any` (problematic in strict mode)
- **Return Type:** `number`
- **Issue:** Violates strict typing; `y` should have explicit type

Part c) Error-Free Function Analysis (3+2 marks)

i) Function Validity and Error Analysis

```

// Code Fragment (i)
(i) function introduce(name:String, IdNumber) {
  console.log("Hello, . name , has ID : IdNumber");
  Intro("Brendan")
}

// Code Fragment (ii)
(ii) function showNames(ames:string[],names:string[]) {

```

```

    console.log(names);
    for(var i = 0; i<names.length;i++) {
        console.log(names[i]);
    }
}

// Called as:
showNames("Ruth","Bob","Guy","Anita");
showNames(["Ruth","Bob","Guy","Anita"]);

```

Analysis of (i) - CONTAINS ERRORS:

Errors Identified:

1. Parameter Type Error: ``typescript

// Code Fragment (i)

```

(i) function introDuce(name:String, IdNumber) {
    console.log("Hello, . name , has ID : IdNumber");
    Intro("Brendan")
}

```

// Code Fragment (ii)

```

(ii) function showNames(ames:string[],names:string[]) {
    console.log(names);
    for(var i = 0; i<names.length;i++) {
        console.log(names[i]);
    }
}

```

// Called as:

```

showNames("Ruth","Bob","Guy","Anita");
showNames(["Ruth","Bob","Guy","Anita"]);

```

****Analysis of (i) - CONTAINS ERRORS:****

****Errors Identified**:**

1. ****Parameter Type Error**:** ``IdNumber`` lacks type annotation (should be ``: number`` or ``: string``)
2. ****String Template Error**:** Missing backticks; should use template literal: `` `Hello, ${name}, has ID: ${IdNumber}` ``
3. ****Incorrect String Syntax**:** ``"Hello, . name , has ID : IdNumber"`` doesn't interpolate variables
4. ****Function Call Error**:** ``Intro("Brendan")`` should be ``introDuce("Brendan", someId)`` with both required parameters
5. ****Missing Parameter**:** ``introDuce()`` requires two parameters but ``Intro()`` provides only one

****Corrected Version**:**

```

``typescript
function introDuce(name: string, IdNumber: number): void {
    console.log(`Hello, ${name}, has ID: ${IdNumber}`);
}

introDuce("Brendan", 12345);

```


Analysis of (ii) - MIXED VALIDITY:**Issues:**

1. **Parameter Mismatch:** Function expects array parameter: ```typescript`

```
function introDuce(name: string, IdNumber: number): void {
  console.log(Hello, ${name}, has ID: ${IdNumber});
}
```

```
introDuce("Brendan", 12345);
```

```
**Analysis of (ii) - MIXED VALIDITY:**
```

```
**Issues**:
```

1. ****Parameter Mismatch**:** Function expects array parameter: ``introDuce(ames: string[], names: string[])``
2. ****First Call Error**:** ``showNames("Ruth","Bob","Guy","Anita")`` passes individual strings, not array
3. ****Second Call Correct**:** ``showNames(["Ruth","Bob","Guy","Anita"])`` passes array correctly
4. ****Logic Issue**:** Function expects TWO array parameters but receives one

```
**Corrected Version**:
```

```
``typescript
function showNames(names: string[]): void {
  console.log(names);
  for (let i = 0; i < names.length; i++) {
    console.log(names[i]);
  }
}
```

```
// Correct calls:
```

```
showNames(["Ruth", "Bob", "Guy", "Anita"]);
```

Part d) Code with Dimensions Interface (4+3 marks)**i) & ii) Missing Code Replacements for ```typescript`**

```
function showNames(names: string[]): void {
  console.log(names);
  for (let i = 0; i < names.length; i++) {
    console.log(names[i]);
  }
}
```

```
// Correct calls:
```

```
showNames(["Ruth", "Bob", "Guy", "Anita"]);
```

```
---
```

```
### Part d) Code with Dimensions Interface (4+3 marks)
```

```
#### i) & ii) Missing Code Replacements for ``typescript
```

```

**Original Code Structure**:
``typescript
interface Dimensions {
  xPos: number;
  yPos: number;
}

function printDimensions(dims: Dimensions): void {
  console.log("***^***");    // Replace with print statement
  console.log("***^***");    // Replace with print statement
}

```

****Code (d-i) - Replacement for ``typescript**

```

interface Dimensions {
  xPos: number;
  yPos: number;
}

```

```

function printDimensions(dims: Dimensions): void {
  console.log("^"); // Replace with print statement
  console.log("^"); // Replace with print statement
}

```

****Code (d-i) - Replacement for `\*\*\*^` (statements for xPos and yPos):****

****Option 1** - Using template literals:**

```

``typescript
function printDimensions(dims: Dimensions): void {
  console.log(`x = ${dims.xPos}`); // First ^**
  console.log(`y = ${dims.yPos}`); // Second ^**
}

```

Option 2 - Using string concatenation:

```

function printDimensions(dims: Dimensions): void {
  console.log("x = " + dims.xPos);
  console.log("y = " + dims.yPos);
}

```

Option 3 - Combined output:

```

function printDimensions(dims: Dimensions): void {
  console.log(`x = ${dims.xPos}, y = ${dims.yPos}`);
}

```

iii) Replacement for ``typescript

```

function printDimensions(dims: Dimensions): void {
  console.log(x = ${dims.xPos}); // First ^**

```

```
console.log(y = ${dims.yPos}); // Second ^**
}
```

```
**Option 2** - Using string concatenation:
```typescript
function printDimensions(dims: Dimensions): void {
 console.log("x = " + dims.xPos);
 console.log("y = " + dims.yPos);
}
```

**Option 3** - Combined output:

```
function printDimensions(dims: Dimensions): void {
 console.log(`x = ${dims.xPos}, y = ${dims.yPos}`);
}
```

**iii) Replacement for \*\*B\*\* - Display yPos Value**

**Code:**

```
const val where val is the value of yPos

// Meaning: where val is yPos value
```

**Replacement:**

```
console.log(`y = ${dims.yPos}`); // Displays yPos value
```

Or with explicit variable assignment:

```
const yPosValue = dims.yPos;
console.log(`y = ${yPosValue}`);
```

**iv) Replacement for ```typescript**

const val where val is the value of yPos

// Meaning: where val is yPos value

```
Replacement:
```typescript
console.log(`y = ${dims.yPos}`); // Displays yPos value
```

Or with explicit variable assignment:

```
const yPosValue = dims.yPos;
console.log(`y = ${yPosValue}`);
```

iv) Replacement for ****C**** - Display yPos Value

Same as (iii) for consistency:

```
console.log(`y = ${dims.yPos}`);
```

Code (d-ii) - Alternative Implementation

Original Code (d-ii):

```
interface Dimensions {
  xPos: number;
  yPos: number;
}

function printDimensions(dims: Dimensions): void {
  console.log("***^***");
  console.log("***^***");
}
```

For a Dimensions object with ````typescript`
`console.log(y = ${dims.yPos});`

```
---

#### Code (d-ii) - Alternative Implementation

**Original Code (d-ii)**:
```typescript
interface Dimensions {
 xPos: number;
 yPos: number;
}

function printDimensions(dims: Dimensions): void {
 console.log("***^***");
 console.log("***^***");
}
```

For a Dimensions object with `xPos = 3` and `yPos = 4`:

v) Replacement for **\*\*B\*\*** - xPos value:

```
console.log(`x = ${dims.xPos}`); // Output: x = 3
```

**\*\*vi)** Replacement for ``typescript

```
console.log(x = ${dims.xPos}); // Output: x = 3
```

```
**vi) Replacement for `**C**` - yPos value:
``typescript
console.log(`y = ${dims.yPos}`); // Output: y = 4
```

## QUESTION 3: Angular Framework (25 Marks)

### Part a) Angular Application Files (4 marks)

When creating a new Angular application, the files shown in the right-side directory structure are generated. The roles of each file:

1. **``typescript**

```
console.log(y = ${dims.yPos}); // Output: y = 4
```

---

**## QUESTION 3: Angular Framework (25 Marks)**

**### Part a) Angular Application Files (4 marks)**

When creating a new Angular application, the files shown in the right-side directory structure are generated. The roles of each file:

1. **``app.component.css``** - Component-specific styling
  - Contains CSS rules applied only to this component
  - Uses view encapsulation to prevent style leakage
  - Scoped to component template elements
  - Alternative: inline styles or `styleUrls`` array
2. **``app.component.html``** - Component template
  - Defines the HTML view/UI for the component
  - Can use Angular directives (`*ngIf``, `*ngFor``, etc.)
  - Can bind to component properties and events
  - Can include other components
3. **``app.component.ts``** - Component class
  - Defines component logic and state
  - Contains properties, methods, lifecycle hooks
  - Decorated with `@Component`` decorator
  - Exports component class
4. **``styles.css``** - Global styles
  - Applied to entire application
  - Not scoped to individual components
  - Used for global CSS resets, utilities, themes
  - Referenced in `angular.json`` configuration

```

Part b) Component Analysis (2+1+1+2 = 6 marks)

i) Role of Component Metadata

```typescript
@Component({
  selector: 'app-root',
  imports: [],
  template: `<h1>Default</h1>`,
  styleUrls: ['./app.component.css'],
})

```

Metadata Explanation (last 3 lines of file):

```

1. ```typescript
  @Component({
    selector: 'app-root',
    imports: [],
    template: <h1>Default</h1>,
    styleUrls: ['./app.component.css'],
  })

```

Metadata Explanation (last 3 lines of file):

1. **selector: 'app-root'**
 - CSS selector for component
 - Defines HTML tag name when component is used: `<app-root></app-root>`
 - Used in `index.html` to bootstrap application
 - Convention: prefix with `app-` (`app-root`, `app-home`, etc.)
2. **imports: []**
 - Array of Angular modules/components to import
 - Registers dependencies needed by this component's template
 - Example: `imports: [CommonModule, FormsModule]`
 - Empty array means no external modules required
3. **template**
 - Inline HTML template (alternative to `templateUrl`)
 - Contains component's view
 - Can use Angular directives and data binding
 - Short templates suitable for inline; longer ones in separate file
4. **styleUrls: ['./app.component.css']**
 - Array of external stylesheet paths
 - Paths relative to component file
 - Styles are view-encapsulated (scoped to component)
 - Multiple files can be included: `['style1.css', 'style2.css']`

ii) Complete Explanation (max 2 sentences)

Combined Explanation:

The metadata object in the `@Component` decorator provides Angular with essential configuration for the component. It specifies the component's CSS selector (`'app-root'`),

```
required imports (`CommonModule`), HTML template (`<h1>Default</h1>`), and component-scoped stylesheets (`app.component.css`), allowing Angular to properly initialize, render, and style the component in the application.
```

```
---
```

Part c) Home Component Creation (2 marks)

If a new component called `**`home`**` is created:

i) Directory Name

src/app/home/

ii) Component's TypeScript File Name

home.component.ts

```
#### iii) Component Class Name (in .ts file)
```typescript
export class HomeComponent { }
```

#### iv) Component's Selector Value (metadata)

```
@Component({
 selector: 'app-home',
 ...
})
```

#### Explanation:

- Directory follows naming convention: lowercase, component name
- TypeScript file: `````  
src/app/home/

#### ii) Component's TypeScript File Name

home.component.ts

```
iii) Component Class Name (in .ts file)
```typescript
export class HomeComponent { }
```

iv) Component's Selector Value (metadata)

```
@Component({
  selector: 'app-home',
  ...
})
```

Explanation:

- Directory follows naming convention: lowercase, component name
- TypeScript file: `{component-name}.component.ts`
- Class name: PascalCase with "Component" suffix (`HomeComponent`)
- Selector: lowercase with `app-` prefix (`app-home`)

Part d) Making Home Component Accessible (6 marks)

To make the home component accessible within the app component, implement the following:

i) The Import Statement

In `app.component.ts`:

```
import { HomeComponent } from './home/home.component';
```

Purpose:

- Imports HomeComponent class from home module
 - Makes component class available for registration
 - Required before adding to `typescript`
- ```
import { HomeComponent } from './home/home.component';
```

```
Purpose:
- Imports HomeComponent class from home module
- Makes component class available for registration
- Required before adding to `imports` array

ii) Modification to Add Component to Imports Metadata

Original:
```typescript
@Component({
  selector: 'app-root',
  imports: [],
  template: `<h1>Default</h1>`,
  ...
})
```

Updated:


```
@Component({
  selector: 'app-root',
  imports: [HomeComponent], // Add HomeComponent here
  template: `<h1>Default</h1><app-home></app-home>`,
  ...
})
```

iii) Add Home Component's View to App Component's View

```
**In ``typescript
@Component({
  selector: 'app-root',
  imports: [],
  template: <h1>Default</h1>,
  ...
})
```

```
**Updated**:
```

```
``typescript
@Component({
  selector: 'app-root',
  imports: [HomeComponent], // Add HomeComponent here
  template: `<h1>Default</h1><app-home></app-home>`,
  ...
})
```

iii) Add Home Component's View to App Component's View

In **app.component.html** or inline template:

```
<h1>Default</h1>
<app-home></app-home>
```

Alternative (inline template in decorator):

```
@Component({
  selector: 'app-root',
  imports: [HomeComponent],
  template: `
    <h1>Default</h1>
    <app-home></app-home>
  `,
})
```

Explanation:

- Use component selector (``html

Default

...

Alternative (inline template in decorator):

```
@Component({
  selector: 'app-root',
  imports: [HomeComponent],
  template: `
    <h1>Default</h1>
    <app-home></app-home>
  `,
})
```

Explanation:

- Use component selector (`app-home`) as HTML tag
- Component renders where tag is placed
- HomeComponent must be in imports for Angular to recognize it

Part e) HomeLocation Interface Integration (3 marks)

Given Interface:

```
export interface HomeLocation {
  id: number;
  street: string;
  town: string;
  region: string;
}
```

i) Import Statement (1 mark)

```
**In ``typescript
export interface HomeLocation {
  id: number;
  street: string;
  town: string;
  region: string;
}
```

i) Import Statement (1 mark)

```
**In `app.component.ts` or `home.component.ts`:
``typescript
import { HomeLocation } from './home/homeLocation';
```

ii) Making HomeLocation Member of AppComponent (1 mark)

```

**In ```typescript
import { HomeLocation } from './home/homeLocation';

```

ii) Making HomeLocation Member of AppComponent (1 mark)

```

**In `app.component.ts`:
```typescript
export class AppComponent {
 homeLocation: HomeLocation;

 constructor() {
 this.homeLocation = {
 id: 50,
 street: '123 Main Street',
 town: 'Buea',
 region: 'Southwest'
 };
 }
}

```

iii) Assigning Value to HomeLocation Member (1 mark)

**Complete Implementation:**

```

export class AppComponent {
 homeLocation: HomeLocation = {
 id: 50,
 street: '123 Main Street',
 town: 'Buea',
 region: 'Southwest'
 };
}

```

**In Template** (```typescript  
export class AppComponent {  
homeLocation: HomeLocation;

```

constructor() {
this.homeLocation = {
id: 50,
street: '123 Main Street',
town: 'Buea',
region: 'Southwest'
};
}
}

```

#### iii) Assigning Value to HomeLocation Member (1 mark)

```

Complete Implementation:
```typescript

```

```
export class AppComponent {
  homeLocation: HomeLocation = {
    id: 50,
    street: '123 Main Street',
    town: 'Buea',
    region: 'Southwest'
  };
}
```

In Template (`app.component.html`):

```
<div>
  <p>Location ID: {{ homeLocation.id }}</p>
  <p>Street: {{ homeLocation.street }}</p>
  <p>Town: {{ homeLocation.town }}</p>
  <p>Region: {{ homeLocation.region }}</p>
</div>
```

Filled Answer for Question:

The blank asking for information about an object with id = 50 in Street 20 of Buea's SW Region:

```
{
  id: 50,
  street: '20 Street',
  town: 'Buea',
  region: 'Southwest'
}
```

Part f) Code Fragment Analysis (5 marks)

For Code Fragment (a)

```
<app-housing-location
  *ngFor="let housingLocation of hosingLocationList"
  [housingLocation]="housingLocation">
</app-housing-location>
```

i) Deduce the Select Metadata for Embedded Component

```
selector: 'app-housing-location'
```

ii) HousingLocation Value Reference (in RHS expression)

```
[housingLocation]="housingLocation"
```

This refers to the current ``html

Location ID: {{ homeLocation.id }}

Street: {{ homeLocation.street }}

Town: {{ homeLocation.town }}

Region: {{ homeLocation.region }}

...

Filled Answer for Question:

The blank asking for information about an object with id = 50 in Street 20 of Buea's SW Region:

```
{
  id: 50,
  street: '20 Street',
  town: 'Buea',
  region: 'Southwest'
}
```

Part f) Code Fragment Analysis (5 marks)

For Code Fragment (a)

```
<app-housing-location
  *ngFor="let housingLocation of hosingLocationList"
  [hosingLocation]="hosingLocation">
</app-housing-location>
```

i) Deduce the Select Metadata for Embedded Component

```
selector: 'app-housing-location'
```

ii) HousingLocation Value Reference (in RHS expression)

```
[hosingLocation]="hosingLocation"
```

This refers to the current `hosingLocation` object from the `*ngFor` loop iteration.

iii) Rendering Description

When the loop executes, the template renders a separate `app-housing-location` component instance for each element in `hosingLocationList`. Each instance receives one housing location object through the `hosingLocation` property binding.

For Code Fragment (b)

```
<ul>
  <li *ngFor="let hero of heroes">
    <button type="button" (click)="selectHero(hero.name)">
      {{ hero.name }}
    </button>
  </li>
</ul>
```

iv) Assume Heroes Array Structure

```
heroes: Array<{name: string; [key: string]: any}> = [
  { name: "Hero1" },
  { name: "Hero2" },
  // ... more heroes
];
```

Explanation: Heroes is an array of objects, each containing at least a ``typescript

- `{{ hero.name }}`

...

iv) Assume Heroes Array Structure

```
heroes: Array<{name: string; [key: string]: any}> = [
  { name: "Hero1" },
  { name: "Hero2" },
  // ... more heroes
];
```

Explanation: Heroes is an array of objects, each containing at least a `name` property of type string.

v) Identify Binding Types (3 types for both fragments)

For both code fragments, these binding types exist:

1. Property Binding (`[property]="expression"`)

- Fragment (a): `[housingLocation]="housingLocation"` - binds component input
- Sends data from parent to child component

2. Event Binding (`((event))="method()"`)

- Fragment (b): `(click)="selectHero(hero.name)"` - handles click events
- Sends data/actions from template to component

3. Interpolation (`{{ expression }}`)

- Fragment (a): Implicit (could add `{{ housingLocation.address }}`)
- Fragment (b): `{{ hero.name }}` - displays data in template

vi) Explain Each Binding Type

Property Binding - `[housingLocation]="housingLocation"`:

- Sends the current `housingLocation` object from the parent component to the child `app-housing-location` component
- Data flows one-way: parent → child
- Child component uses `@Input()` decorator to receive the value

Event Binding - `(click)="selectHero(hero.name)"`:

- Listens for click events on the button element
- When clicked, calls component method `selectHero()` with the hero's name
- Data flows one-way: template → component
- Used for user interactions

Interpolation - `{{ hero.name }}`:

- Displays component property value in template
- Angular evaluates the expression and converts to string
- One-way binding: component → template
- Updates automatically when property changes

SUMMARY TABLE: Key Concepts

Topic	Key Points
XML	Well-formed vs. Valid; DTD vs. Schema; Element nesting; Attribute handling
TypeScript	Transpilation to JavaScript; Type annotations; Function signatures; Interface usage
Angular	Components; Metadata; Selectors; Data binding; Imports; Directives

Master's Level Notes:

- These topics form the foundation of modern web development
- XML remains critical for data interchange and configuration
- TypeScript's type system prevents runtime errors at scale
- Angular's component model enables scalable, maintainable applications
- Integration of these technologies creates robust enterprise systems